

SIMATS ENGINEERING

ASSESSMENT TOOL 2

R.HARIKA(192572)

COURSE NAME: Database Management Systems
COURSE CODE: CSA0501

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Activity Tool : Industry Problem Solving Task (Medium) Assessment Tool Weightage: 35%

Dr

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	An e-commerce company needs to store 10 million product records. Recommend a file organization strategy and justify your choice with cost analysis.	BL4 – Analyze	5
2	A banking system requires fast retrieval of transactions by account number and date range. Design an indexing strategy with appropriate index types.	BL4 – Analyze	5
3	A healthcare system stores patient records accessed both by PatientID and by doctor name. Design a multi-level indexing solution.	BL4 – Analyze	5
4	A logistics company needs to efficiently handle dynamic insertions and deletions of shipment records. Analyze whether B-Tree or B+ Tree is more suitable.	BL4 – Analyze	5
5	Design a hashing scheme for an HR system with 5000 employees. Evaluate static hashing vs. extendible hashing for your scenario.	BL3 – Apply	5

6	An airline reservation system must support point queries by flight number and range queries by date. Propose a combined index structure.	BL4 – Analyze	5
7	For a university student database, design the file organization and secondary indexing to support fast queries by department and CGPA range.	BL4 – Analyze	5
8	Analyze the disk block utilization for Heap, Sorted, and Hashed file organizations given 100,000 records with 50 bytes each and 4KB blocks.	BL4 – Analyze	5
9	A social media platform needs to index 500 million posts by user_id, timestamp, and hashtag. Design a composite indexing strategy.	BL4 – Analyze	5
10	Compare the performance (insert, delete, search time) of Heap File, Sorted File, and Hashed File for a supermarket billing system with 1 million daily transactions.	BL4 – Analyze	5

Code of Conduct:

I RASINENI HARIKA(192572140) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

R. Harika

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context

Application of	25%	Applies advanced and relevant	Applies appropriate concepts	Limited application of concepts	Incorrect or no
----------------	-----	-------------------------------	------------------------------	---------------------------------	-----------------

Engineering Concepts		concepts effectively to solve the problem	with minor errors		application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. E-commerce: 10 million product records

Recommended: Use a **B+ Tree-based indexed file organization**, with data stored in pages/blocks and an index on frequently searched attributes such as ProductID, category, and price.

Justification:

- 10 million records are too large for an in-memory structure.
- B+ Trees provide **$O(\log N)$** search, insertion, and deletion.
- Leaf nodes are linked, making range queries such as price ranges efficient.
- Records can be stored in blocks to reduce disk I/O.

Cost analysis:

- A sequential scan may require reading almost all 10 million records for a search → very high I/O cost.
- A B+ Tree requires only a small number of index-page accesses, approximately $O(\log_b N)$.
- Extra storage is required for the index, but the faster search usually justifies this cost.

Conclusion: B+ Tree indexing gives a good balance of storage cost and search performance for a 10-million-record database.

2. Banking: Transactions by account number and date range

Use a **composite B+ Tree index**:

(AccountNumber, TransactionDate)

Why?

- AccountNumber is the primary search attribute.
- TransactionDate supports efficient range queries.
- Example:

WHERE AccountNo = 101 AND TransactionDate BETWEEN '2026-01-01' AND '2026-01-31'

The index first locates the account and then scans the required date range.

Index structure:

- Root → internal nodes → leaf nodes.
- Leaf nodes contain pointers to transaction records.
- Since B+ Tree leaves are linked, date-range retrieval is efficient.

Additional index: If queries frequently search only by date, create a separate B+ Tree on TransactionDate.

Conclusion: A composite B+ Tree (AccountNumber, TransactionDate) is the most appropriate primary index.

3. Healthcare: PatientID and doctor name

Use a **multi-level indexing solution** with separate indexes.

Primary index

Create a **B+ Tree on PatientID**.

PatientID → Patient Record

PatientID is usually unique, so retrieval is very fast.

Secondary index

Create a **B+ Tree on DoctorName**.

DoctorName → List of Patient Record Pointers

Because several patients may belong to the same doctor, this is a **non-unique secondary index**.

Multi-level structure

Root Index → Intermediate Index → Leaf Index → Records

The upper levels reduce the number of disk accesses needed to locate the required leaf.

Conclusion: A primary B+ Tree on PatientID plus a secondary B+ Tree on DoctorName provides efficient access through both search keys.

4. Logistics: B-Tree vs B+ Tree

Feature	B-Tree	B+ Tree
Records in internal nodes	Possible	No
Records in leaf nodes	Yes	Yes
Range queries	Good	Excellent
Sequential access	Less efficient	Very efficient
Fan-out	Lower	Higher
Dynamic insertion/deletion	Good	Excellent

For a logistics company, shipment records are frequently **inserted, deleted, and searched**.

B+ Tree is preferable because:

- All records are stored at leaf level.
- Leaf nodes are linked for sequential/range access.
- Higher fan-out reduces tree height.
- Splitting and merging maintain balance automatically.
- Dynamic updates do not require reorganizing the complete file.

Conclusion: Choose a **B+ Tree** because it provides efficient dynamic updates and excellent range/sequential access.

5. HR system: 5000 employees

A **hashing scheme** can use EmployeeID as the hash key.

Example:

$$h(\text{EmployeeID}) = \text{EmployeeID} \bmod 100$$

This initially creates 100 buckets.

Static hashing

- Fixed number of buckets.
- Very simple and fast.
- Average search is close to **O(1)**.
- However, if the number of employees increases significantly, overflow buckets may develop.
- Overflow reduces performance.

Extendible hashing

- Uses a directory and dynamically expands buckets.
- Buckets split when they become full.
- Handles employee additions efficiently.
- Avoids long overflow chains.

Factor	Static Hashing	Extendible Hashing
Initial simplicity	High	Moderate
Growth handling	Poor	Excellent
Overflow	More likely	Low
Dynamic resizing	No	Yes
Search	O(1) average	O(1) average

Conclusion: For 5000 employees with predictable size, static hashing is sufficient. If the HR database is expected to grow substantially, **extendible hashing is better**.

6. Airline reservation: flight number and date

Use a **combined indexing structure**:

Composite B+ Tree

Create:

(FlightNumber, FlightDate)

This supports:

- Exact search by flight number.
- Date-range searches for a particular flight.
- Efficient ordered traversal of dates.

Example:

FlightNumber = AI101 AND FlightDate BETWEEN June 1 AND June 30

The index first finds AI101, then scans its date range.

Additional index

If queries frequently search:

FlightDate BETWEEN X AND Y

without specifying flight number, create a separate B+ Tree on FlightDate.

Conclusion: A composite B+ Tree (FlightNumber, FlightDate) plus an optional date index provides efficient point and range queries.

7. University database: department and CGPA

File organization

Use a **sorted file organization based on Department** or a clustered organization by department.

Example:

CSE records → ECE records → EEE records → ...

This groups students from the same department together.

Secondary index

Create a **B+ Tree index on (Department, CGPA)**.

Example query:

Department = 'CSE' AND CGPA BETWEEN 8.0 AND 9.0

The index:

1. Locates the CSE section.
2. Finds $CGPA \geq 8.0$.
3. Sequentially scans until $CGPA > 9.0$.

Advantages:

- Fast department lookup.
- Efficient CGPA range search.
- Linked B+ Tree leaves make range retrieval efficient.

Conclusion: Cluster records by department and use a composite B+ Tree (Department, CGPA).

8. Disk block utilization

Given:

- Records = **100,000**
- Record size = **50 bytes**
- Block size = **4 KB = 4096 bytes**

Blocking factor

So, each block stores **81 records**.

Number of blocks

Total data size:

Comparison

Organization	Blocks	Utilization/Comments
Heap	1235	High; records inserted wherever space exists
Sorted	1235	High; records kept in sorted order
Hashed	$\approx 1235 + \text{overflow}$	Depends on hash distribution

Approximate utilization:

Conclusion: All three require about **1235 data blocks** under ideal packing. Hashed files may require additional overflow blocks because of collisions.

9. Social media: 500 million posts

The system needs searches by:

- user_id
- timestamp
- hashtag

A suitable strategy is **multiple composite B+ Tree indexes**.

Index 1

(user_id, timestamp)

Supports:

WHERE user_id = X ORDER BY timestamp
and date/time-range queries for a particular user.

Index 2

(hashtag, timestamp)

Supports:

WHERE hashtag = '#AI' AND timestamp BETWEEN X AND Y

Optional Index 3

(timestamp)

Useful for global time-based queries.

Considerations

500 million records produce very large indexes, so:

- Partition data by time, such as monthly partitions.
- Use B+ Trees within partitions.
- Consider separate storage/indexes for frequently accessed recent posts.

Conclusion: Composite indexes (user_id, timestamp) and (hashtag, timestamp), combined with time partitioning, provide scalable query performance.

10. Supermarket billing: 1 million daily transactions

Compare **Heap**, **Sorted**, and **Hashed** files.

Operation	Heap File	Sorted File	Hashed File
Search	$O(N)$	$O(\log N)$ with binary search/index	$O(1)$ average
Insert	$O(1)$ average	$O(N)$	$O(1)$ average
Delete	$O(N)$ search + deletion	$O(N)$	$O(1)$ average
Range search	Poor	Excellent	Poor
Maintenance	Easy	Expensive	Moderate

Heap File

- New transaction can be appended quickly.
- Excellent for high-volume insertion.
- Searching a particular transaction may require scanning many records.

Sorted File

- Very fast searching and excellent range queries.
- Insertions can be expensive because records may need to be shifted.
- Not ideal for 1 million new transactions every day.

Hashed File

- Excellent for exact searches such as TransactionID.
- Average insertion and search are **$O(1)$** .
- Collision handling is required.
- Poor for range queries.

Conclusion: For a supermarket billing system dominated by **transaction insertion and exact TransactionID searches**, **Hashed File is the best choice**. If frequent date-range reports are required, a secondary B+ Tree index on transaction date should be added.